

BABEŞ–BOLYAI TUDOMÁNYEGYETEM KOLOZSVÁR
MATEMATIKA ÉS INFORMATIKA KAR
INFORMATIKA SZAK

Projektjelentés

Intelligens Otthoni Energiarendszerek Optimalizálása Természetből Inspirált Metaheurisztikák Alkalmazásával



TÉMAVEZETŐ:
<TÉMAVEZETŐ NEVE>

SZERZŐ:
<HALLGATÓ NEVE>

BABEȘ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Project Report

**Optimization of Smart Home Energy
Systems with Nature-Inspired Metaheuristics**



ADVISOR:

<SUPERVISOR NAME>

AUTHOR:

<STUDENT NAME>

UNIVERSITATEA BABEȘ–BOLYAI, CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ

Raport de proiect

Optimizarea Sistemelor de Energie pentru Locuințe prin Metaheuristici Inspirare din Natură



CONDUCĂTOR ȘTIINȚIFIC:

<NUMELE CONDUCĂTORULUI>

ABSOLVENT:

<NUMELE
STUDENTULUI>

BABEŞ–BOLYAI UNIVERSITY OF CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND INFORMATICS
SPECIALIZATION: COMPUTER SCIENCE

Project Report

Optimization of Smart Home Energy Systems with Nature-Inspired Metaheuristics

Abstract

A dolgozat angol nyelvű rövid összefoglalója (kitöltendő).

This work is the result of my own activity. I have never provided or received unauthorised assistance in this work.

2026

<STUDENT NAME>

ADVISOR:

<SUPERVISOR NAME>

Tartalomjegyzék

1. fejezet

Bevezetés és Problémafelvetés

***Összefoglaló:** A fejezet bemutatja a HEMS célját és a keresletoldali szabályozás jelentőségét.*

A globális energetikai átalakulás, a decentralizált termelés térnyerése és a fenntarthatósági elvárások a Smart Grid technológiák felé terelik a villamosenergia-hálózatokat. A HEMS kulcsszereplő a lakossági oldalon: kétirányú kommunikációt és energiaáramlást kezel, összehangolja a fogyasztókat, tárolást és megújuló termelést.

1.1. A projekt célja

A cél egy olyan HEMS optimalizálási modell kidolgozása, amely képes:

- háztartási eszközök összehangolt vezérlésére,
- megújuló források optimális kihasználására,
- energiatárolók intelligens irányítására,
- villamosenergia-költség minimalizálására,
- hálózati terhelési csúcsok csökkentésére (PAR redukció),
- felhasználói komfort biztosítására.

1.2. A keresletoldali szabályozás (DSM) jelentősége

Terheléseltolás (Load Shifting) előnyei:

1. Közvetlen költségmegtakarítás: 20–40%

1. FEJEZET: BEVEZETÉS ÉS PROBLÉMAFELVETÉS

2. Hálózati csúcsok csökkentése
3. Fosszilis erőművek igénybevételének mérséklése
4. Károsanyag-kibocsátás csökkentése

2. fejezet

Alkalmazott Algoritmusok

Összefoglaló: Négy metaheurisztikát és egy hibrid eljárást alkalmaztunk: GA, PSO, GWO és GA-PSO hibrid.

2.1. Genetikus Algoritmus (GA)

Fő jellemzők:

- **Előnyök:** jól kezeli diszkrét változókat, robusztus.
- **Hátrányok:** lassabb konvergencia.
- **Teljesítmény:** $\approx 15\text{--}20\%$ költségcsökkenés.

Implementált GA pseudokód:

1. Start: N db véletlen egyed generálása (populáció).
2. Fitness: minden egyed költségének kiszámítása.
3. Ciklus 1.. G :
 - Elitizmus: a legjobb 2 egyed automatikus továbbjutása.
 - Szelekció: szülők Tournament módszerrel.
 - Keresztezés: aritmetikai átlag + zaj.
 - Mutáció: Gauss-eloszlású véletlen módosítás.
 - Csere: új populáció, határok ellenőrzése.
4. Vége: a legjobb megoldás visszaadása.

2.2. Particle Swarm Optimization (PSO)

Sebesség-frissítési egyenlet:

$$V_i^{k+1} = w \cdot V_i^k + c_1 r_1 (P_{best,i} - X_i^k) + c_2 r_2 (G_{best} - X_i^k) \quad (2.1)$$

Pozíció-frissítés:

$$X_i^{k+1} = X_i^k + V_i^{k+1} \quad (2.2)$$

Előny: gyors, egyszerű. Hátrány: lokális optimumra hajlamos. Teljesítmény: $\approx 15\text{--}18\%$ költségcsökkenés.

2.3. Grey Wolf Optimizer (GWO)

Hierarchikus struktúra: $\alpha, \beta, \delta, \omega$. Pozíció-frissítés:

$$\vec{D}_\alpha = |\vec{C}_1 \cdot \vec{X}_\alpha - \vec{X}| \quad (2.3)$$

$$\vec{X}_1 = \vec{X}_\alpha - \vec{A}_1 \cdot \vec{D}_\alpha \quad (2.4)$$

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3} \quad (2.5)$$

Előny: kevesebb paraméter, jobb exploration-exploitation egyensúly. Teljesítmény: $\approx 18\text{--}20\%$ költségcsökkenés.

2.4. Hibrid GA-PSO

Logika:

1. PSO sebesség-frissítés: gyors keresés.
2. GA keresztezés és mutáció: diverzifikáció.
3. Adaptív váltás a két fázis között.

Teljesítmény: $\approx 18\text{--}22\%$ költségcsökkenés.

3. fejezet

Kísérleti Eredmények

***Összefoglaló:** Három scenárióban (tél, nyár, borús) 20-20 futással értékeltük az algoritmusokat; a nyers eredmények CSV/JSON formában mentve vannak.*

3.1. Kísérleti scenáriók

Három eltérő scenárióban futtattuk a kísérleteket (20 futás/algoritmus/scenárió), hogy a robusztusságot mérjük:

1. **Téli:** alacsony PV termelés, magasabb alapfogyasztás, nagy hálózati függés.
2. **Nyári:** magas PV termelés (csúcsban 6 kW), alacsonyabb fogyasztás, jelentős visszatáplálási potenciál.
3. **Viharos/Borús (cloudy):** ingadozó PV termelés, magas zaj, az akkumulátor dinamikus kezelése szükséges (stresszteszt).

Reprodukálhatóság és mentések: minden futás determinisztikus bázis-seeddal (42) indul, a 20 ismétléshez futásonként eltolva; a nyers eredményeket és futási időket listáit CSV/JSON formában mentjük: `results_winter.(csv|json)`, `results_summer.(csv|json)`, `results_cloudy.(csv|json)`.

3.2. Téli scenárió eredményei

Az alapvonal (akku és időzítés nélkül) télen átlagosan **42.09 RON/nap**.

3. FEJEZET: KÍSÉRLETI EREDMÉNYEK

3.1. táblázat. Téli Szenárió statisztikái (20 futás, 50/100-as populáció, 150 iteráció, globális seed: 42)

Algoritmus	Átlag költség	Legjobb futás	Átlagos idő (s)
GA (P=50)	37.05	34.92	0.31
GA (P=100)	37.25	35.06	0.62
PSO	36.87	34.56	0.21
GWO	35.40	33.91	0.44
Hybrid	36.28	34.42	0.24

3.3. Nyári Szenárió eredményei

Az alapvonal (akku és időzítés nélkül) nyáron átlagosan **12.48 RON/nap**.

3.2. táblázat. Nyári Szenárió statisztikái (20 futás, 50/100-as populáció, 150 iteráció, globális seed: 42)

Algoritmus	Átlag költség	Legjobb futás	Átlagos idő (s)
GA (P=50)	4.08	3.74	0.31
GA (P=100)	3.93	3.64	0.62
PSO	6.67	5.20	0.20
GWO	5.17	4.21	0.43
Hybrid	5.88	4.93	0.25

3.4. Viharos/Borús Szenárió eredményei

3.3. táblázat. Viharos/Borús Nap Szenárió statisztikái (20 futás, 50/100-as populáció, 150 iteráció, globális seed: 42)

Algoritmus	Átlag költség	Legjobb futás	Átlagos idő (s)
GA (P=50)	14.13	13.58	0.31
GA (P=100)	13.92	13.60	0.61
PSO	17.08	14.07	0.20
GWO	15.48	14.72	0.43
Hybrid	16.19	14.50	0.25

3.5. Eredmények elemzése

- **Szezonális hatás:** nyáron a magas PV miatt a napi költség drasztikusan csökkenthető (67% megtakarítás, 12.48 $\rightarrow$ 3.93 RON); télen kb. 18% megtakarítás (42.09 $\rightarrow$ 34.42 RON). A borús nap köztes állapot.
- **Algoritmusok:** GA (P=100) robusztus a komplex, változékony esetekben; GWO télen adta a legjobb átlagot; PSO gyors, de nagy keresési térben hajlamos beragadni; a hibrid kiegyensúlyozott, de a többletidő nem mindig hoz jobb optimumot.
- **Reprodukálhatóság:** 20 ismétlés, determinisztikus seed; futási idők 0.2–0.9 s tartományban.

3.6. Ábrák

A generált ábrák: `generated_day_*.png`, `results_convergence_*.png`, `results_boxplot_*.png` (winter, summer, cloudy). A konvergencia görbék mutatják a GA lassabb, de stabilabb, és a PSO gyors, de korai stagnálását; a boxplot-ok a 20 futás statisztikai eloszlását ábrázolják.

4. fejezet

Következtetések és Ajánlások

4.1. Főbb megállapítások

1. **Szezonális különbség:** nyáron az optimalizáció kulcsa a többlettermelés menedzselése (akkutöltés vs. visszatáplálás), télen a drága csúcsidő minimalizálása. A három scenárió (tél, nyár, borús) lefedi a valós üzemeltetési körülményeket.
2. **Algoritmus választás:** nincs egyetlen "legjobb" algoritmus. A GA ($P=100$) robusztus a komplex, változékony esetekben, míg a GWO hatékonyabb a szigorúbb korlátokkal terhelt (téli) környezetben.
3. **Akkumulátor szerepe:** a borús tesztet bizonyította az ESS fontosságát a termelés ingadozásainak kisimításában.
4. **Futási teljesítmény:** 0.21–0.92 s közötti futási idők; a PSO a leggyorsabb, de kompromisszum a megoldás minőségében.

4.2. Gazdasági megtérülés

4.1. táblázat. Beruházási és megtérülési kalkuláció becslés

Tétel	Költség/Érték
HEMS hardver	≈ 1200 RON
5 kWp napelem rendszer	≈ 15000 RON
10 kWh akkumulátor	≈ 18000 RON
Teljes beruházás	≈ 34200 RON
Éves megtakarítás (becsült)	≈ 6200 RON
Megtérülési idő	≈ 5.5 év

Támogatással (Casa Verde) a megtérülés 3–4 évre csökkenthető.

4.3. Környezeti hatás

- 6000 kWh saját PV termelés $\rightarrow 4.8$ tonna CO₂/év
- 2000 kWh terhelés-eltolás és hatékonyság $\rightarrow 1.6$ tonna CO₂/év
- **Összesen: ≈ 6.4 tonna CO₂/év megtakarítás**

4.4. Jövőbeli kutatási irányok

4.4.1. Energiaközösségek (REC)

Peer-to-peer energia kereskedés, közösségi akkumulátor; potenciális további megtakarítás 10–15%.

4.4.2. Multi-objective optimalizálás

Pareto-front alapú megközelítés: felhasználó választhat költség-komfort görbéről, adaptív súlyozással.

4. FEJEZET: KÖVETKEZTETÉSEK ÉS AJÁNLÁSOK

4.4.3. Gépi tanulás integráció

LSTM időjárás előrejelzés, felhasználói szokások tanulása, prediktív karbantartás.

4.4.4. V2G integráció

Elektromos autók mint mobil akkumulátorok: 60 kWh EV kapacitás → 6x nagyobb tárolás, 50–100 RON/hó extra bevétel.

4.5. Implementációs ajánlások

4.5.1. Kis háztartásoknak (< 3 fő)

- **Minimál konfiguráció:** HEMS controller + intelligens konnektorok
- **Költség:** 800–1200 RON
- **Megtakarítás:** 15–20% ($\approx$ 500–800 RON/év)
- **Megtérülés:** 1.5–2 év

4.5.2. Közepes háztartásoknak (3–5 fő)

- **Ajánlott:** HEMS + 3–5 kWp PV + 5–10 kWh akkumulátor
- **Költség:** 25000–35000 RON (támogatással csökkenthető)
- **Megtakarítás:** 30–50% ($\approx$ 3000–5000 RON/év)
- **Megtérülés:** 3–5 év (támogatással), 6–8 év (támogatás nélkül)

A. függelék

Fontosabb programkódok listája

A.1. HEMS kódrendszer

- `codes/main.py`: szcenáriók futtatása (winter, summer, cloudy), eredmények mentése CSV/JSON-be, ábrák generálása.
- `codes/algorithms.py`: GA, PSO, GWO és hibrid GA-PSO implementációja.
- `codes/hems_problem.py`: SmartHomeEnvironment modell, célfüggvény és korlátozás kezelés.
- `codes/data_generator.py`: fogyasztási, PV és árprofil generálása szezononként.

A.2. Mintakódrészlet

Az alábbi példa a hibrid algoritmus fő lépéseit illusztrálja:

```
1      # Visszatesszük az eliteket
      pop = np.vstack((elites, np.clip(offspring, self.lb, self.ub)))
      fitness = np.array([self.func(ind) for ind in pop])

      current_best = np.min(fitness)
6      if current_best < best_fit:
          best_fit = current_best
          best_sol = pop[np.argmin(fitness)].copy()
          self.history.append(best_fit)

11     return best_sol, best_fit

# --- 2. PSO (Változatlan) ---
class PSO(AlgorithmBase):
    def solve(self):
16         w, c1, c2 = 0.6, 1.8, 1.8
         X = np.random.uniform(self.lb, self.ub, (self.pop_size, self.dim))
         V = np.zeros_like(X)
```

A. FÜGGELÉK: FONTOSABB PROGRAMKÓDOK LISTÁJA

```

21     P_best = X.copy()
    P_best_fit = np.array([self.func(x) for x in X])

    g_best = P_best[np.argmin(P_best_fit)].copy()
    g_best_fit = np.min(P_best_fit)
    self.history = [g_best_fit]

26     for _ in range(self.max_iter):
        r1, r2 = np.random.rand(self.pop_size, self.dim), np.random.
            rand(self.pop_size, self.dim)
        V = w * V + c1 * r1 * (P_best - X) + c2 * r2 * (g_best - X)
        X = np.clip(X + V, self.lb, self.ub)

31     fits = np.array([self.func(x) for x in X])
    better = fits < P_best_fit
    P_best[better] = X[better]
    P_best_fit[better] = fits[better]

36     current_min = np.min(fits)
    if current_min < g_best_fit:
        g_best_fit = current_min
        g_best = X[np.argmin(fits)].copy()
    self.history.append(g_best_fit)

41     return g_best, g_best_fit

# --- 3. GWO (Változatlan) ---
class GWO(AlgorithmBase):
46     def solve(self):
        wolves = np.random.uniform(self.lb, self.ub, (self.pop_size, self.
            dim))
        fits = np.array([self.func(w) for w in wolves])

        # Kezdő rendezés
51     idx = np.argsort(fits)
    alpha, beta, delta = wolves[idx[:3]].copy()

    # Elitizmus
    best_sol = alpha.copy()

```

Irodalomjegyzék

- Boyd, S. P. és Vandenberghe, L. *Convex Optimization*. Cambridge University Press, Cambridge, UK, 2004. URL <http://www.stanford.edu/~boyd/cvxbook.html>.
- Nabney, I. T. *NETLAB - Algorithms for Pattern Recognition*. Springer, 2002.